

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 10%

Mock Interview Questions

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.

Answer:

```
IMPORT os, glob, cv2
```

```
FROM concurrent.futures IMPORT ThreadPoolExecuto
```

```
FUNCTION preprocess_image(file_path):
```

```
    # Safe read
```

```
    img = cv2.imread(file_path)
```

```
    IF img IS NULL:
```

```
        RETURN None, file_path # Handle corrupted file
```

```
    # Standardize image dims to lower memory pressure
```

```
    img_resized = cv2.resize(img, (640, 480))
```

```
    # Core Processing: Denoise and CLAHE Contrast Enhancement
```

```
    lab = cv2.cvtColor(img_resized, cv2.COLOR_BGR2LAB)
```

```
    l_channel, a, b = cv2.split(lab)
```

```

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))

cl = clahe.apply(l_channel)

processed_lab = cv2.merge((cl, a, b))

processed_bgr = cv2.cvtColor(processed_lab, cv2.COLOR_LAB2BGR)

RETURN processed_bgr, file_path

FUNCTION batch_processing_pipeline(folder_path):

    image_paths = glob.glob(folder_path + "/*.jpg") + glob.glob(folder_path + "/*.png")

    # ThreadPool for asynchronous parallel I/O and preprocessing

    WITH ThreadPoolExecutor(max_workers=4) AS executor:

        results = executor.map(preprocess_image, image_paths)

    FOR processed_img, path IN results:

        IF processed_img IS NOT NULL:

            cv2.imshow("Batch Pipeline View", processed_img)

            key = cv2.waitKey(100) # Non-blocking delay for GUI rendering

            IF key == 27: BREAK # ESC to exit cleanly

            cv2.destroyAllWindows()

```

Design Justifications & Optimizations

- **Multithreading (I/O Decoupling):** Disk reads (cv2.imread) are I/O bound. Executing file parsing across parallel threads prevents CPU stalls.
- **Colorspace Normalization (CLAHE in LAB):** Modifying contrast on the L channel prevents color shift distortions commonly caused by operating directly in standard BGR/RGB spaces.
- **Pre-allocation & Resize:** Standardizing dimensions (640x480) prevents out-of-memory errors on large uncompressed high-resolution datasets.
- **Key Potential Challenges:** Disk thrashing on spinning hard drives (mitigate by setting worker thread counts based on disk throughput) and variable color profiles (handled by strict colorspace conversions).

2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.

Answer:

When `cv2.VideoCapture` fails to read or decode a stream, systematic isolation is required to locate the bottleneck across hardware, codec support, file corruption, or backend compatibility.

Structured Debugging Steps

- **Source & API Verification:** Test if the file path exists using native OS modules before passing to OpenCV. Ensure the OpenCV installation includes compiled FFmpeg backends (`cv2.getBuildInformation()`).
 - **Container vs. Codec Inspection:** OpenCV may open the container file (.mp4, .mkv) but fail to decode the underlying stream (H.265, AV1) if the required hardware acceleration or codec flags are absent.
 - **Hardware Acceleration Driver Overrides:** Pass explicit API preferences such as `cv2.CAP_FFMPEG` or `cv2.CAP_GSTREAMER` to prevent fallback to incompatible legacy video frameworks.
4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

Answer:

Standard linear contrast adjustments ($I_{out} = \alpha I + \beta$) multiply global pixel intensities, causing bright regions to saturate into severe highlight clipping while noise in dark regions becomes magnified.

Proposed Pipeline Architecture

Raw Dark Image $\longrightarrow$ Convert to LAB $\longrightarrow$ CLAHE (L-Channel) $\longrightarrow$ Denoise (Non-Local Means) $\longrightarrow$ Gamma Correction $\longrightarrow$ Reconvert to BGR

FUNCTION `enhance_dark_image(img)`:

1. Convert to LAB color space to decouple luminance from chromaticity

`lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)`

`l, a, b = cv2.split(lab)`

2. Contrast Limited Adaptive Histogram Equalization (CLAHE)

Operates on local sub-tiles to elevate shadow detail without over-exposing highlights

`clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))`

`l_enhanced = clahe.apply(l)`

```

# 3. Non-Local Means Denoising on the enhanced luminance channel
# Eliminates high-frequency noise amplified by equalization
l_denoised = cv2.fastNlMeansDenoising(l_enhanced, None, h=7,
templateWindowSize=7, searchWindowSize=21)

# 4. Non-linear Gamma Correction ( $\gamma < 1.0$ ) to boost mid-tones
gamma = 0.6
inv_gamma = 1.0 / gamma
table = ARRAY([(i / 255.0) ** inv_gamma * 255 FOR i IN RANGE(256)],
dtype='uint8')
l_gamma = cv2.LUT(l_denoised, table)

# Reconstruct final image
merged = cv2.merge((l_gamma, a, b))
RETURN cv2.cvtColor(merged, cv2.COLOR_LAB2BGR)

```

Image & Video Processing

6. You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy.

Answer:

Decoding video sequentially to process every N -th frame creates severe CPU processing bottlenecks if every frame is fully uncompressed before being skipped.

```

FUNCTION process_every_nth_frame(video_path, N=5):
    cap = cv2.VideoCapture(video_path)
    frame_idx = 0

    WHILE cap.isOpened():
        # Optimization: Grab frame without full decoding step
        grabbed = cap.grab()
        IF NOT grabbed: BREAK

        IF frame_idx % N == 0:
            # Decode frame only when processing criteria is met
            ret, frame = cap.retrieve()
            IF NOT ret: BREAK

            # Execute downstream computer vision tasks
            run_heavy_inference(frame)

        frame_idx += 1

```

cap.release()

Justification & Performance Considerations

- **Memory & Throughput:** cap.grab() fast-forwards the video decoder to the next frame header without running full YUV-to-RGB color space conversions or pixel decoding. cap.retrieve() is called exclusively on the 5th frame, saving up to 70% of frame-decoding overhead.
 - **Accuracy Implications:** In tracking or motion estimation pipelines, skipping too many frames ($N > 10$) causes severe trajectory loss due to large displacement between consecutive bounding boxes. For object detection tasks, $N = 5$ (at 30 FPS = 6 Hz sampling) yields optimal throughput while retaining inter-frame spatial overlap.
8. You need to extract frames from a video and save only those with significant changes. Analyze the problem and design a method, explaining your decision criteria and approach.

Answer:

To extract frames containing significant changes while filtering ambient video jitter, temporal frame differencing combined with structural similarity metrics provides an efficient filtering stage.

```
FUNCTION extract_key_frames(video_path, change_threshold=0.05):
```

```
    cap = cv2.VideoCapture(video_path)
```

```
    ret, prev_frame = cap.read()
```

```
    IF NOT ret: RETURN
```

```
    prev_gray = cv2.cvtColor(cv2.GaussianBlur(prev_frame, (9, 9), 0),  
cv2.COLOR_BGR2GRAY)
```

```
    saved_count = 0
```

```
    WHILE cap.isOpened():
```

```
        ret, frame = cap.read()
```

```
        IF NOT ret: BREAK
```

```
        curr_gray = cv2.cvtColor(cv2.GaussianBlur(frame, (9, 9), 0),  
cv2.COLOR_BGR2GRAY)
```

```
        # Calculate Absolute Frame Difference
```

```
        diff = cv2.absdiff(curr_gray, prev_gray)
```

```
        _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
```

```
        # Calculate ratio of changed pixels relative to total image area
```

```
        non_zero_count = cv2.countNonZero(thresh)
```

```
        total_pixels = frame.shape[0] * frame.shape[1]
```

```

change_ratio = non_zero_count / FLOAT(total_pixels)

# Decision Criterion
IF change_ratio > change_threshold:
    cv2.imwrite("keyframe_" + STR(saved_count) + ".jpg", frame)
    saved_count += 1
    prev_gray = curr_gray # Update baseline frame

cap.release()

```

Decision Criteria & Trade-Offs

- **Gaussian Blurring Kernel:** Pre-blurring prevents camera sensor noise, compression artifacts, and minor lighting flickers from triggering keyframe writes.
- **Dynamic Baseline Updating:** Updating *prev_gray* *only* when a new keyframe is saved ensures slow, gradual movements accumulate over time to properly capture key events.

Feature Detection & Drawing

9. You are asked to highlight key features in an image. Design a feature detection and visualization approach, explaining the techniques and their suitability.

Answer:

Implementation Pseudocode (ORB vs Shi-Tomasi)

FUNCTION extract_and_draw_features(img, technique="ORB"):

```

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    output_img = img.copy()

    IF technique == "SHI_TOMASI":

        # Best for tracked corner features

        corners = cv2.goodFeaturesToTrack(gray, maxCorners=100, qualityLevel=0.01,
            minDistance=10)

        corners = INT_0(corners)

        FOR i IN corners:

            x, y = i.ravel()

            cv2.circle(output_img, (x, y), 4, (0, 0, 255), -1)

```

ELSE IF technique == "ORB":

Best for invariant feature matching

orb = cv2.ORB_create(nfeatures=500)

keypoints, descriptors = orb.detectAndCompute(gray, None)

Draw rich keypoints displaying scale and orientation angles

output_img = cv2.drawKeypoints(img, keypoints, None, color=(0, 255, 0),

flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

RETURN output_img

Feature Detector	Computation Speed	Rotation Invariance	Scale Invariance	Primary Use Case
Shi-Tomasi (Corners)	Extremely High	No	No	Real-time optical flow tracking
ORB (Oriented FAST & Rotated BRIEF)	High (Real-time CPU)	Yes	Yes	Embedded systems, SLAM, matching
SIFT (Scale-Invariant Feature Transform)	Low (Heavy CPU)	Yes	Highly Robust	High-accuracy alignment, 3D reconstruction

Face & Expression Recognition

13. You need to build a simple face recognition demo. Design a minimal working pipeline, explaining each stage and your design decisions.

Answer:

Input Frame —> Haar/DNN Detection —> Align/Crop —> Extract Embeddings (LBPH/FaceNet) —> Cosine/Euclidean Matching

```

FUNCTION face_recognition_pipeline(frame, database_embeddings):
    # Stage 1: Face Detection
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)

    FOR (x, y, w, h) IN faces:
        face_roi = gray[y:y+h, x:x+w]
        face_resized = cv2.resize(face_roi, (100, 100))

        # Stage 2: Feature Extraction using Local Binary Patterns Histograms (LBPH)
        lbph_model = cv2.face.LBPHFaceRecognizer_create()
        label, confidence = lbph_model.predict(face_resized)

        # Stage 3: Decision Thresholding
        IF confidence < 50: # Lower distance score implies higher similarity
            name = database_embeddings.get(label, "Unknown")
        ELSE:
            name = "Unknown"

        # Stage 4: Visualization
        cv2.rectangle(frame, (x,y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(frame, name, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8,
(0,255,0), 2)

    RETURN frame

```

14. A facial expression system fails with slight head movements. Analyze the limitation and propose modifications, justifying your approach.

Answer:

Limitation Analysis: Rigid bounding boxes fail under head rotation (yaw, pitch, roll) because the relative spatial alignment of facial landmarks changes. This spatial distortion degrades traditional classifier performance.

Proposed Modifications

1. **Facial Landmark Alignment (Affine Registration):** Detect eye center coordinates (E_{left} , E_{right}) and compute the angle θ relative to the horizontal axis. Rotate the source image around its center to align the eyes horizontally before cropping the facial ROI.
2. **Perspective Warp Normalization:** Apply a 2D Perspective Transformation Matrix to map non-planar face angles onto a standardized frontal template.

```

FUNCTION align_and_crop_face(image, left_eye, right_eye):

```



```

# Compute angle between eye centers

dY = right_eye.y - left_eye.y

dX = right_eye.x - left_eye.x

angle = ATAN2(dY, dX) * (180 / PI)


# Calculate scale difference relative to target eye distance

dist = SQRT((dX ** 2) + (dY ** 2))

desired_dist = 50.0

scale = desired_dist / dist


# Compute center point between eyes

eye_center = ((left_eye.x + right_eye.x) // 2, (left_eye.y + right_eye.y) // 2)


# Get 2D Rotation Matrix

M = cv2.getRotationMatrix2D(eye_center, angle, scale)


# Apply Affine Transformation to align face upright

aligned = cv2.warpAffine(image, M, (image.width, image.height), flags=cv2.INTER_CUBIC)

RETURN aligned

```

Code of Conduct:

*I DARANI E (192312120) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weight age	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively

Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately